Aircraft Engine Remaining Useful Life Prediction

Estimating remaining operating cycles from engine sensor history · NASA C-MAPSS FD001 turbofan dataset

Prateek Shakya · Yogesh Modi · Kapish Rohilla · Abhishek Kumar Patel · Vikas Kumar

1 THE PROBLEM

RUL = final recorded cycle − current cycle

Example: failure at cycle 200, now at cycle 150



200 − 150 = 50 cycles of life remain

- Predictive maintenance: detect degradation before failure.
- Supports planning with an estimated remaining-cycle value.
- RUL is counted in operating cycles, not days.

2 THE DATASET

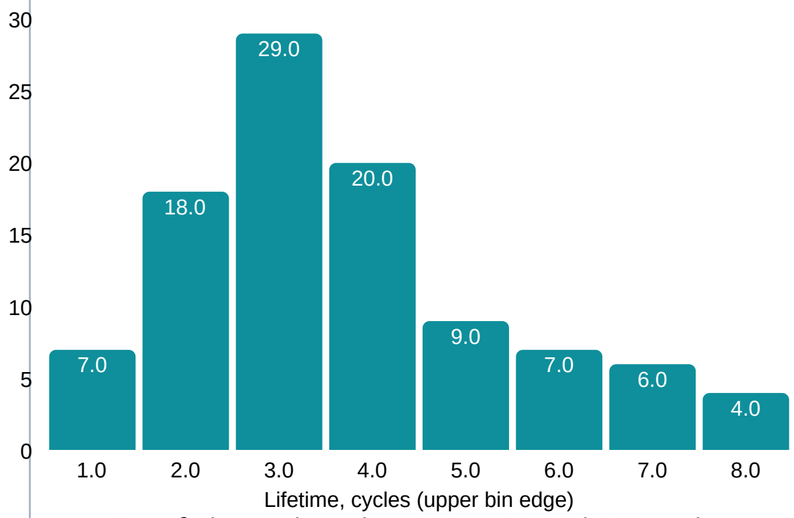
20,631
training rows
100 engines

13,096
test rows
100 engines

26
columns
incl. 21 sensors

0
missing values
data quality check

- One row = one engine at one operating cycle.
- Training engines run to failure, so RUL labels can be calculated.
- Test engines stop early; true RUL is supplied separately.



Engines fail anywhere between 128 and 362 cycles (mean 206), so RUL cannot be read from the cycle count alone; it must be inferred from sensor trends.

Fig. 1 *Lifetime of the 100 training engines (cycles).*

Exploration: which sensors carry the degradation signal?

Engine 1 of the training set and all 100 training engines, using RUL computed from each engine's final cycle.

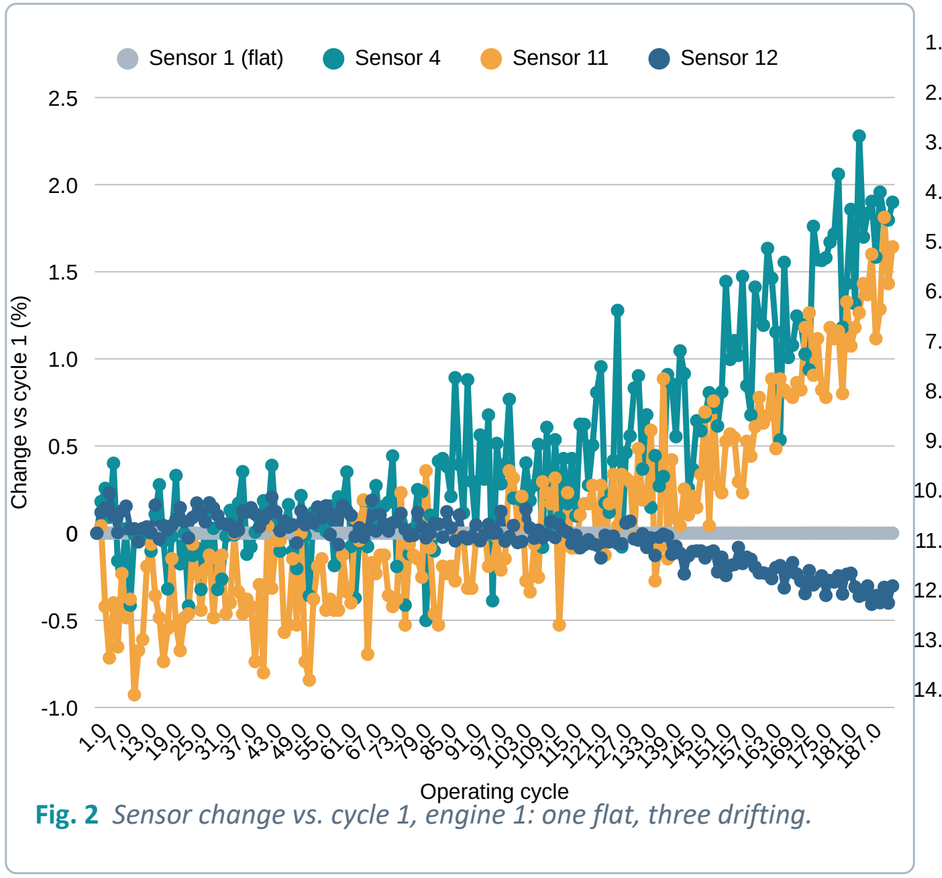


Fig. 2 *Sensor change vs. cycle 1, engine 1: one flat, three drifting.*

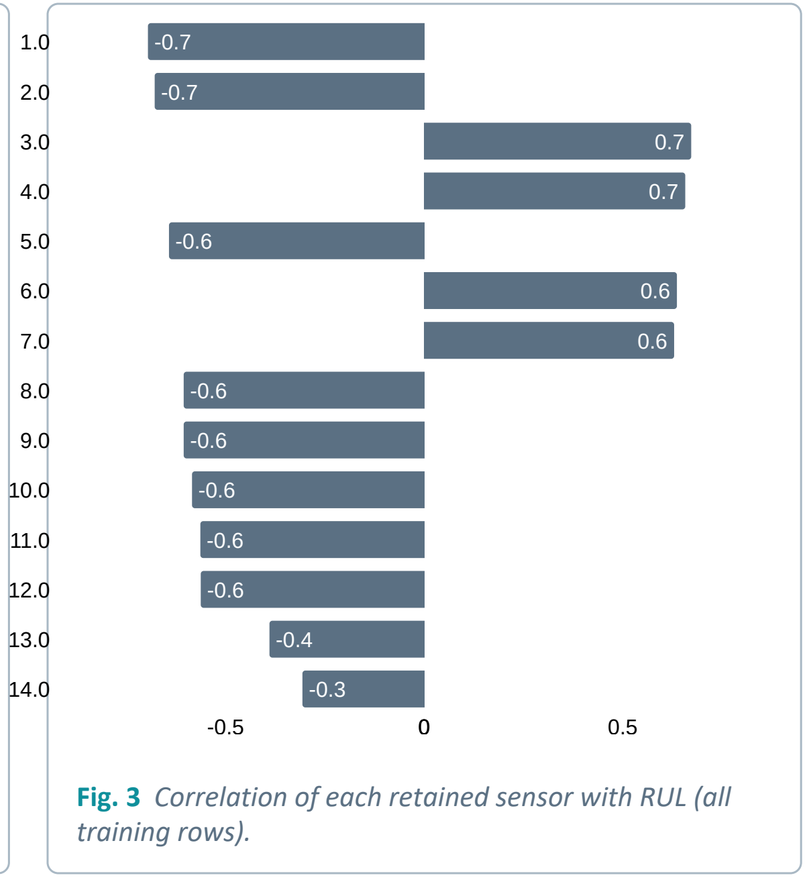


Fig. 3 *Correlation of each retained sensor with RUL (all training rows).*

WHAT THE DATA SHOWS

- 8 flat columns removed: setting_3 and sensors 1, 5, 6, 10, 16, 18, 19.
- No missing values in 20,631 rows.
- Sensors 4 and 11 rise and sensor 12 falls as failure nears.
- Correlations with RUL are only moderate (|r| 0.31 to 0.70).

KEY TAKEAWAY Flat columns carry no signal. The 14 retained sensors drift with wear but are noisy, so single readings are not enough: trend features are needed.

Feature engineering: summarising recent sensor behaviour

Rolling statistics over the last 10 cycles of the same engine (current and past readings only). Example: sensor 11, engine 1.

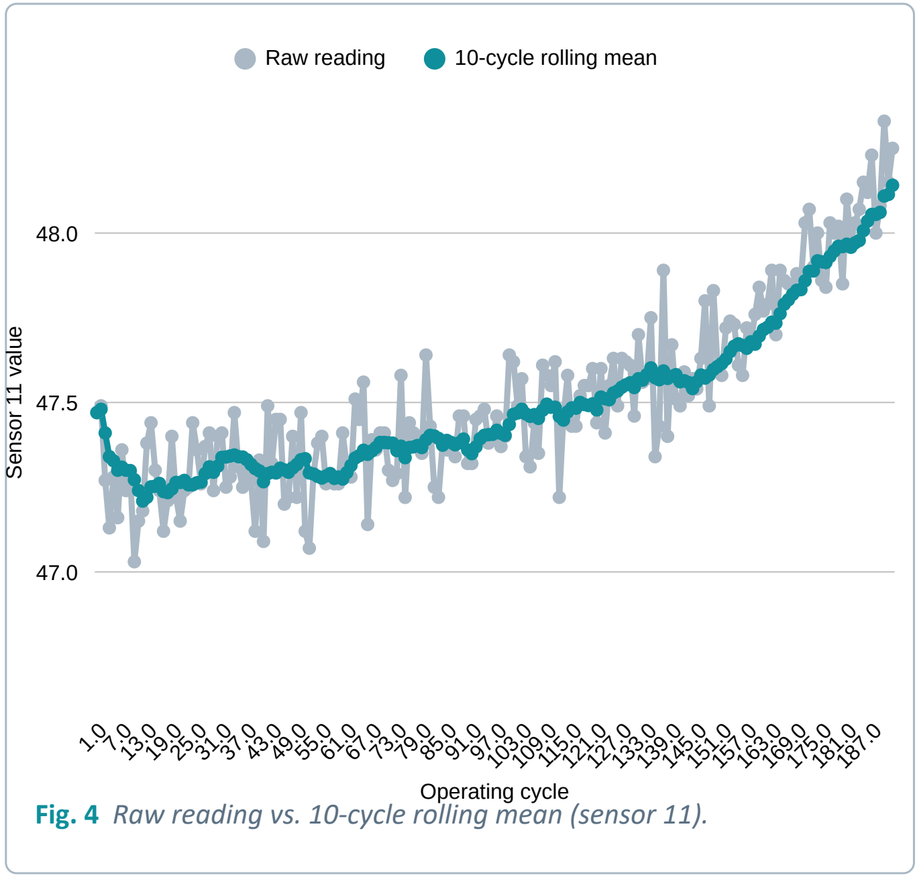


Fig. 4 *Raw reading vs. 10-cycle rolling mean (sensor 11).*

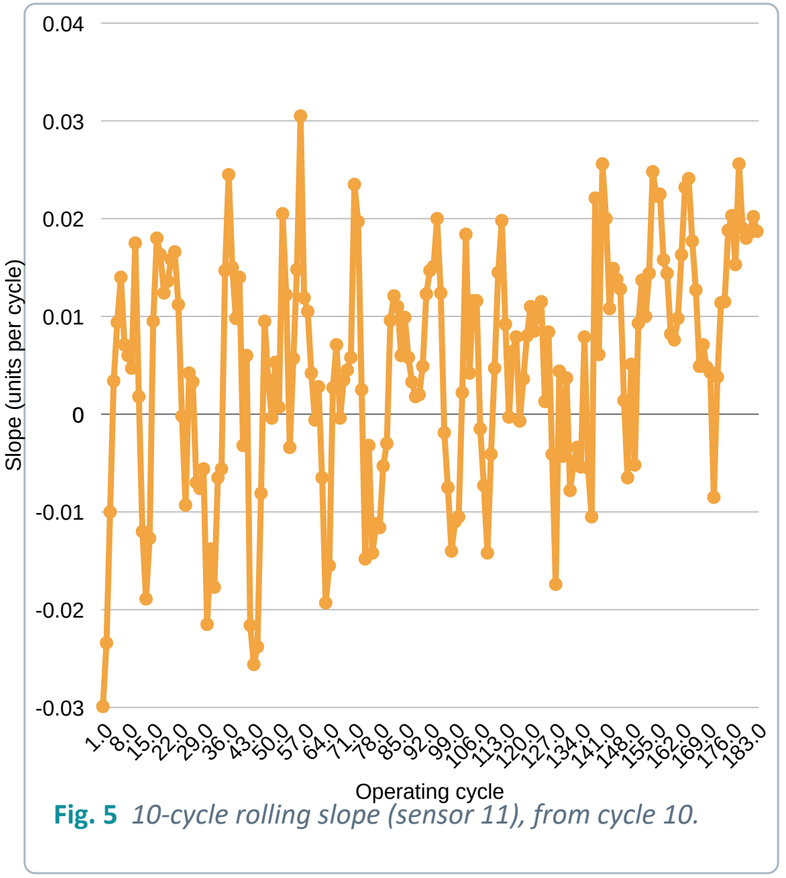


Fig. 5 *10-cycle rolling slope (sensor 11), from cycle 10.*

THE 57 INPUT FEATURES

57 = 1 cycle + 14 raw sensors + 14 × 3 rolling statistics

- Mean: recent average level.
- Std. deviation: recent variability.
- Slope: direction of change.
- Window = current + previous 9 cycles, so no future readings leak in.

KEY TAKEAWAY For sensor 11 the average rolling slope rises from +0.002 in the first third of engine life to +0.009 in the last third: the features expose a trend hidden in noisy raw readings.

Methodology: models and validation design

Same features, same engine-based split and same validation points for every model.

1 Load & check → 2 Create RUL → 3 Explore sensors → 4 Build features → 5 Train models → 6 Evaluate

FOUR REGRESSION MODELS

MODEL	FAMILY	HOW IT WORKS
OLS Linear Regression	Linear baseline	Fits a straight-line relationship between features and RUL.
Elastic Net	Regularised linear	Shrinks less useful coefficients to reduce reliance on single inputs.
Random Forest	Tree ensemble	Averages many decision trees to capture non-linear sensor patterns.
Gradient Boosting	Boosted trees	Adds trees one by one, each correcting the previous errors.

VALIDATION DESIGN

80 engines to train

20 engines to validate



5 cut-offs × 20 engines = 100 validation points

MAE = mean |actual − predicted|

RMSE = √ mean (actual − predicted)²

Errors are in cycles; RMSE penalises large errors more.

KEY TAKEAWAY Splitting by engine (never by row) keeps the same engine out of both training and validation, so all four models are compared fairly on identical points.

Results: non-linear models give the lowest error

Validation scores over the 100 fixed cut-off points, in operating cycles; lower is better.

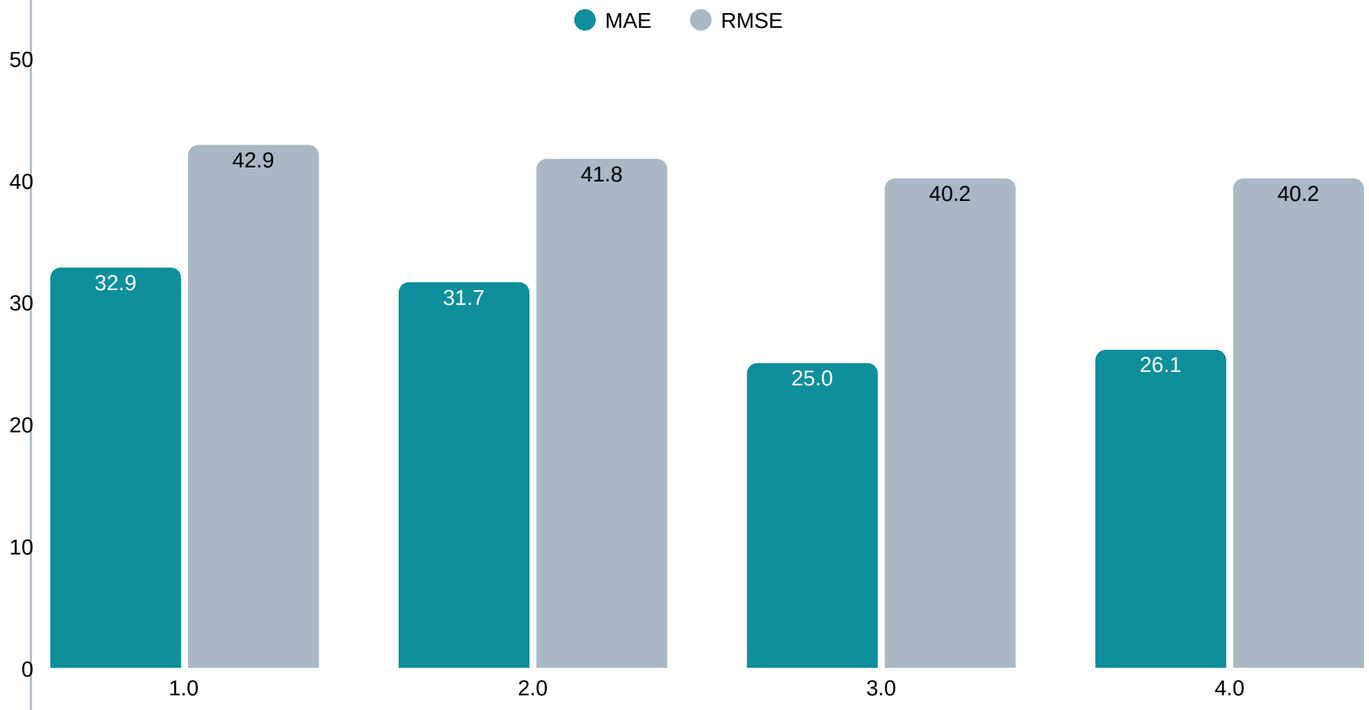


Fig. 6 *Validation MAE and RMSE for the four models.*

SCORECARD

MODEL	MAE	RMSE
OLS Linear Regression	32.86	42.92
Elastic Net	31.66	41.77
Random Forest	**25.03**	**40.17**
Gradient Boosting	26.12	40.17

INTERPRETATION

- Random Forest: lowest MAE (25.03), about 24% below OLS.
- Random Forest and Gradient Boosting tie on RMSE (40.17).
- RMSE far above MAE for every model: a few large errors dominate.

KEY TAKEAWAY Non-linear models cut MAE by about 5 to 8 cycles versus linear baselines. Held-out test MAE so far (linear models only): OLS 24.91, Elastic Net 23.61.

Error analysis, conclusions and next steps

Where do the large errors come from? Linear-model validation predictions, plus the high-RUL subset for all four models.

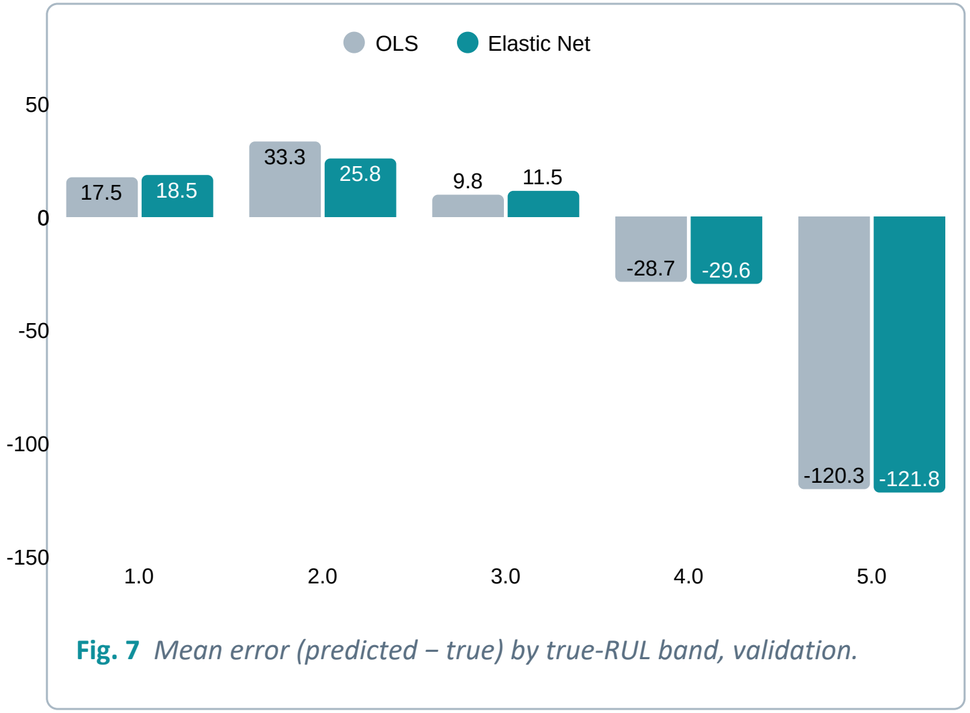


Fig. 7 *Mean error (predicted − true) by true-RUL band, validation.*

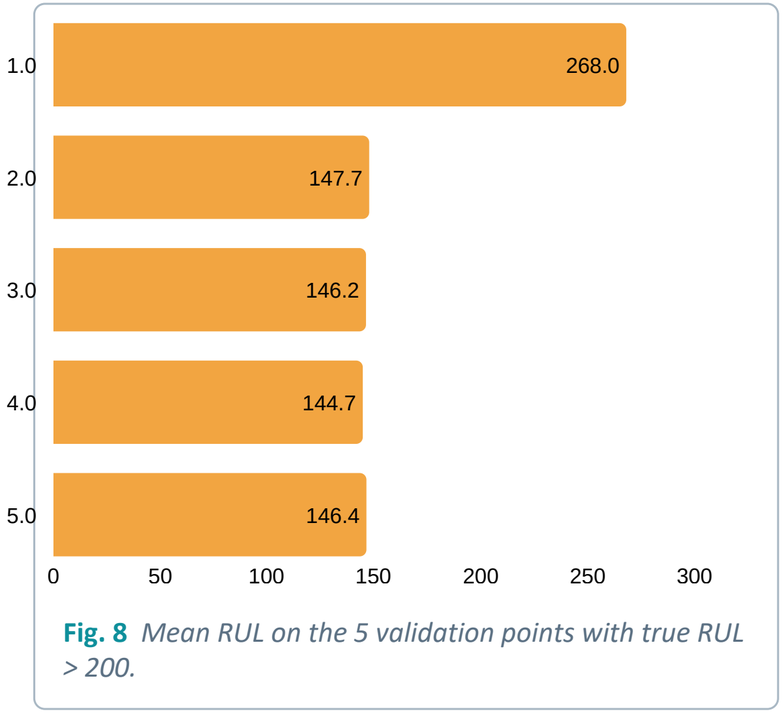


Fig. 8 *Mean RUL on the 5 validation points with true RUL > 200.*

CONCLUSIONS

- Predictions regress toward the middle: too high near failure, too low for long-life engines (about −120 cycles at 200+).
- All four models underpredict the high-RUL subset.
- A 10-cycle window sees only recent behaviour; the subset is small, so read with caution.

NEXT STEPS

1. Complete and verify the held-out test evaluation for all models.
2. Try longer rolling windows and more temporal features.
3. Consider a sequence model such as an LSTM for long sensor histories.

RUL prediction is feasible; reliable estimates require strong features and careful evaluation.

Team introduction

Each member owned one stage of the shared project pipeline.



ENROLLMENT NO.	NAME	ROLE / CONTRIBUTION
2401020048	**Prateek Shakya**	Data loading and EDA: named columns, calculated training RUL, checked data quality, plotted sensor trends, identified eight flat columns and checked correlations.
2401010519	**Yogesh Modi**	Feature engineering: created 10-cycle rolling mean, standard deviation and slope features for 14 sensors, using only current and past readings.
2401010210	**Kapish Rohilla**	Validation and linear models: prepared the engine-based split and MAE/RMSE metrics; trained and evaluated OLS and Elastic Net.
2401010020	**Abhishek Kumar Patel**	Tree-based models: trained and compared Random Forest and Gradient Boosting using the prepared features and validation points.
2401010503	**Vikas Kumar**	Final evaluation: combines predictions, compares validation and test results, analyses early- and late-life errors, and summarises limitations.